

Information Processing 25-26 Group 9

March 23, 2026

1. System Purpose

Our system streams live cryptocurrency market data from Binance, computes microstructure features in real time, and uses an FPGA-accelerated linear regression model to predict the future price differential ratio of an asset over a 2-second horizon. Predicted returns are converted into BUY/SELL trading signals that drive live paper trades, while model weights are published to a cloud REST API for consumption by any connected node.

The design prioritises latency: linear regression is retrained via incremental outer-product accumulation, which maps efficiently to pipelined FPGA logic. A separate FPGA node enables runtime voice control for asset and feature switching.

[Link to video](#)

[Github Repository](#)

2. System Architecture

As per the requirements of the task, the system has two active PYNQ-Z1 boards communicating over a Flask endpoint configured on an AWS cloud server.

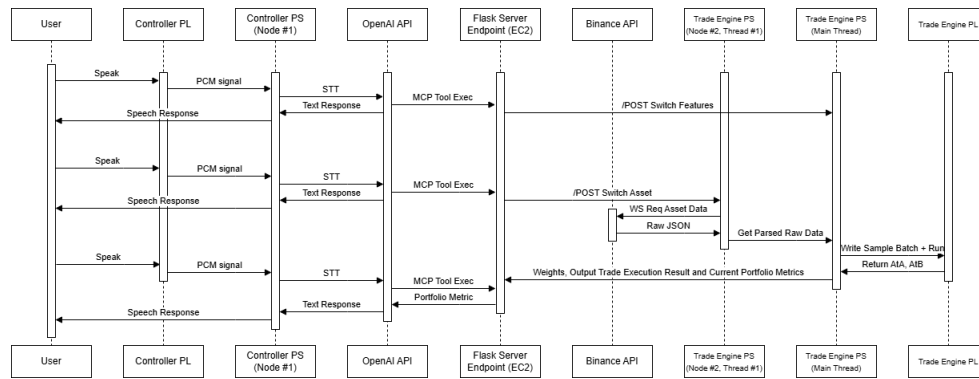


Figure 1: System Architecture Diagram

2.1 PYNQ Controller Node (Node #1)

The controller node provides the user interface; the hardware converts raw 1-bit PDM from the microphone into high-fidelity 16-bit PCM audio at 16 kHz. A custom clock generator drives the microphone at 2.4 MHz, with the resulting stream decimated by a factor of 150 using a CIC Compiler to reach the target 16 kHz sample rate. To preserve signal integrity, the CIC output is maintained at 32-bit precision before passing through a secondary FIR Compiler. This FIR stage acts as an inverse-sinc compensator, flattening the frequency response to correct the inherent passband drop of the CIC filter, resulting in a cleaner audio signal for the Google Speech Recogniser API. Finally, the 32-bit stream is normalised and truncated to 16-bit signed PCM in the software layer for API compatibility.

In software, the audio signal is transcribed via Google’s Speech Recogniser API. The resulting text is sent to the OpenAI API over a persistent websocket connection. Responses are streamed incrementally and buffered, with audio playback only starting once the full response is received. This avoids fragmented speech while maintaining low HTTP request overhead.

User commands, such as asset switching or feature reconfiguration, are forwarded to the cloud server via REST endpoints. The controller node, therefore, acts as an interface layer, isolating user interaction from the trading and compute pipeline.

2.2 Cloud Node (AWS Flask Server)

The cloud node runs a Flask REST API backed by SQLite (`data.db`). Four append-only tables store model weights, portfolio state, instructions, and features. The dashboard polls key endpoints to display the latest regression equation and portfolio metrics.

Endpoints are summarised below, the rationale behind the architectural choices (SQLite, JSON, the thin-coordination pattern) is discussed in Section 3.

Method	Path	Role
POST/GET	/matrix	Latest model: asset , features , weights ; clients derive signals locally.
POST/GET	/weights	Feature names and asset (BTC/ETH).
POST	/store_instruction	Voice command text; server infers asset from keywords.
GET	/instruction	Newest instruction for trading node polling.
POST/GET	/portfolio	Paper-trading balance , accuracy .
GET	/graph, /dashboard	Server-rendered PNG history; live HTML monitor.
POST	/flush	Clear portfolio history (preserves model & instruction records).

Table 1: Summary of Flask REST API Endpoints

Critically, the two PYNQ nodes never communicate directly. Both read from and write to the same cloud state, which keeps the fleet loosely coupled and either board can be restarted, swapped, or pointed at a different asset independently without the other node needing to know.

2.3 PYNQ Trading Engine Node (Node #2)

The trading node performs real-time data ingestion, feature extraction, and model training. A BinanceWSClient thread maintains a persistent WebSocket connection, streaming live trade events and order book updates. Parsed data is fed into the CalculationEngine, which computes seventeen microstructure features every 200 ms.

All features are vectorised using NumPy and z-score normalised before training. Feature vectors are labelled with future log-returns and buffered until a minimum of 4000 samples are reached.

Training is then performed on the programmable logic (PL), where outer-product accumulation ($A^T A$, $A^T b$) is executed with $380 \mu\text{s}$ average throughput. The accumulated results are returned to the processing system (PS), where final weight computation is completed.

Updated weights and portfolio metrics are then sent to the cloud server, closing the end-to-end loop between data ingestion, model training, and trading execution.

3. Design Decisions

3.1 Incremental Normal-Equation Updates

Early discussions explored convolutional neural networks and Markov chain Monte Carlo samplers. Neural networks were ruled out after consulting with the lab supervisor: the PYNQ-Z1’s BRAM is insufficient for the matrix multiplications in neural inference, and retraining would require off-chip DDR access at every step. Following a visit to the Financial Signal Processing laboratory at Imperial, where a PhD student explained that many practical HFT strategies are simple and derive their edge from latency rather than model complexity, we settled on linear regression.

The key mathematical insight that made real-time retraining feasible is that the normal equations update with rank-one corrections. Let $A \in \mathbb{R}^{n \times d}$ and $B \in \mathbb{R}^{n \times 1}$ be the current design and target matrices. Appending one new row $a \in \mathbb{R}^{1 \times d}$ with label b gives:

$$A'^T A' = A^T A + a^T a, \quad A'^T B' = A^T B + a^T b.$$

This means that each retrain requires only one outer product per new sample, not a full matrix multiply over all history. The hardware kernel implements exactly this accumulation, making the FPGA the natural home for the innermost loop.

3.2 Cloud Coordination API

We kept the cloud tier as a thin coordination layer: it stores shared state and relays it between nodes but never runs the model or touches market data. Pushing model logic to the cloud would have added a network round-trip into every prediction and duplicated complexity that already lives on the FPGA.

All state sits in a single SQLite file (**data.db**) with four append-only tables keyed by auto-incrementing IDs and timestamps. Our access pattern is trivial: every endpoint either inserts one row or fetches the most recent via `ORDER BY id DESC LIMIT 1`. The trading node writes to **matrix** and **portfolio**; the controller writes to **instructions** and **weights_data**. This natural separation meant SQLite’s file-level locking was never a bottleneck, and avoided managing a separate database process entirely. Because rows are never overwritten, full history is preserved for debugging and for the dashboard’s portfolio graph. The only destructive operation is **/flush**, which clears **portfolio**.

We ran Flask’s built-in single-threaded WSGI server, which was adequate given the low request rate and millisecond handler times. The database initialises lazily on the first request via a **before_request** hook, so the schema creates itself on first deployment. JSON was chosen over HTTP so both PYNQ nodes could use the same minimal **requests** client code, reserving WebSocket connections for the cases where streaming matters: Binance market data and OpenAI responses.

If the EC2 instance becomes unreachable, both nodes continue local processing: the trading node keeps ingesting data and training, and the controller keeps accepting voice input. The next successful request resynchronises state automatically, validating the loose coupling.

3.3 Fixed vs Floating-Point Representation

Originally, we used `ap.fixed<18,13>` for input features (5 integer bits, 13 fractional bits, resolution ≈ 0.00012) and `ap.fixed<64,40>` for accumulators. When switching to a design based purely on Vivado using SystemVerilog, we first converted the floating-point elements to 16-bit integers in software while using a 64-bit accumulator. We chose these values because they maintain accuracy in our matrix multiplier while allowing exactly 1 DSP slice per multiplication in 1 cycle. Multiplying two 16-bit integers produces a 32-bit result; accommodating accumulation over 4000 samples requires ~ 44 bits. A 64-bit accumulator was therefore used to prevent overflow and simplify software interfacing.

This introduces quantisation error (max ± 0.03125), corresponding to a worst-case $\sim 1.25\%$ relative error. In practice, errors average out across samples and have negligible impact on final weights.

3.4 512-bit AXI4 Data Bus

Each 512-bit memory word carries one complete sample. The hardware extracts all values in a single cycle via bit-field unpacking. This highly pipelined memory extraction removes any serial bottleneck ensuring a $\sim 90\mu\text{s}$ DMA register setup.

3.5 Horizon-Based Labels and Welford Normalisation

The target for each feature vector is the log-return of an asset at time t : $\ln(p_{t+H}/p_t)$ where $H = 10$ steps = 2 s. Labels are assigned only after H real-time steps have elapsed, preventing lookahead bias. Z-score normalisation uses Welford’s single-pass algorithm so that running mean and variance are updated without storing any historical data. Weights are denormalised back to original units before being published to the API.

3.6 WebSocket Communication

Both nodes use persistent WebSocket connections rather than REST polling. On the controller, the OpenAI WebSocket stream is buffered until completion before playback to avoid fragmented speech. On the trading node, the Binance stream feeds directly into the calculation engine, sustaining the 200 ms cycle. Both WebSocket clients run as daemon threads so they terminate automatically on exit. This keeps data ingestion, model computation, and user interaction concurrent without requiring explicit thread management or synchronisation overhead.

3.7 Vectorised Feature Computation

Within the `CalculationEngine`, feature calculations were implemented using NumPy vectorised operations rather than Python for loops. Many of the features implemented in the Trading Engine (volume ratios, price differences, rolling statistics) operate over streams of market data, making them suitable for vectorised operations.

By implementing these computations in NumPy, the heavy numerical work is executed in C under the hood, rather than in the Python interpreter. This significantly reduces overhead and allows all features to be computed within the 200 ms update window without becoming a bottleneck.

Although this optimisation runs on the PYNQ processing system rather than the FPGA fabric, it remains important for overall system performance, as inefficient Python overhead would otherwise dominate latency and negate the benefits of hardware acceleration.

4. Performance Target Metrics

The table below captures performance targets for the four pipeline phases: data ingestion, hardware inference, model quality, and hardware utilisation.

Metric	Target Value	Justification
HW accumulation (4000 samples)	~ 81 ms	Attempt to beat SW-optimised design by $10\times$
SW-optimised batch (4000 rows)	~ 810 ms	Approximate baseline estimation for software calculation time
Quantisation error	$<0.5\%$ relative	This value is an industry standard for machine learning models
Feature scrape period	200 ms	Matches Binance depth update rate
Cloud API latency (GET <code>/matrix</code>)	<150 ms median	Dominated by WAN RTT from London to eu-west-2; 150 ms is comfortably within the 200 ms feature cadence so the cloud is never on the critical path.
<code>/graph</code> response (GET)	<600 ms median	Includes server-side matplotlib render of a ~ 110 kB PNG plus download; we measured empty-portfolio renders at ~ 200 ms and budgeted $3\times$ for a full history.
Speak-to-response latency	~ 2 s	Benchmarked asset-switch responsiveness from other industry models
Directional accuracy (250 iter)	50.01%	Primary model quality signal
Paper trading return	+5% (\$1000 $\rightarrow$ \$1050)	End-to-end profitability baseline
PL clock frequency	100 MHz (10 ns)	Throughput ceiling

Table 2: Table Outlining the Performance Target Metrics of the System

5. Testing Approach

Testing operated at three levels simultaneously.

5.1 Three-Implementation Comparison

The system simultaneously runs an unoptimised software LR (stores full A and B in RAM), a software-optimised LR (accumulates only $A^T A$ and $A^T b$), and the hardware implementation. Running all three on every batch provides a built-in regression test: agreement between the two software variants confirms the normalisation pipeline is correct; agreement between the optimised software and hardware confirms the fixed-point arithmetic introduces no meaningful bias.

On every training cycle, the three LR variants (Hardware, Unoptimised Software, Optimised Software) receive identical normalised input. Their weight vectors are printed side by side. Any divergence between the two software variants indicates a data pipeline bug; divergence between optimised software and hardware points to a fixed-point precision issue. This approach provided immediate, continuous feedback throughout development and detected both the accumulator overflow bug and an early normalisation mismatch before they could corrupt trading results.

Moreover, by measuring the time each engine took to compute the weights, we were able to directly compare performance on the same dataset. This allowed us to clearly quantify the speed-up from the hardware design while ensuring that all implementations produced consistent and accurate results.

5.2 Lookahead-Safe Labelling

Early prototypes inadvertently used future prices as features, inflating accuracy to near 100%. The labelling logic was refactored so that the feature vector at step t is paired with p_{t+H} only after H real polling steps have elapsed and the corresponding price has arrived via the live WebSocket.

5.3 Paper Trading

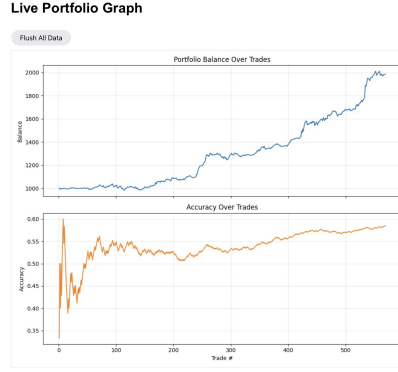


Figure 2: Dashboard of Accuracy and Balance Statistics Over Number of Trades

The **TestingEngine** uses live model weights to place live paper market orders. It tracks directional accuracy, cumulative balance, and PnL separated from the total account equity to isolate this strategy’s contribution. These metrics are also posted to the AWS node to allow the user to visualise such metrics on a dashboard.

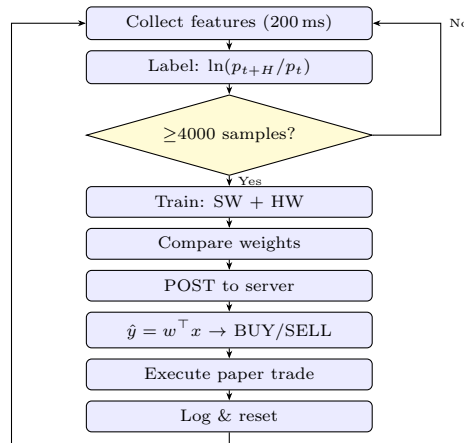


Figure 3: Testing and Execution Flow

The following measurements were made against the performance targets:

Metric	Target Value	Achieved Value
HW accumulation (4000 samples)	~81 ms	~3.49 ms
SW-optimised batch (4000 rows)	~810 ms	~30.4 ms
Quantisation error	<0.5% relative	~1.25%
Feature scrape period	200 ms	200 ms
Cloud API latency (GET / matrix , WAN)	<150 ms median ($n = 50$)	~94 ms (single request)
/ graph response (GET, WAN)	<600 ms median ($n = 50$)	~453 ms median ($n = 50$), range 443–494 ms
Speak-to-response latency	~2 s	~2 s
Directional accuracy (250 iter)	50.01%	57.6%
Paper trading return	+5% (\$1000→\$1050)	+112.6% (\$1000→\$2126) at 20x leverage
PL clock frequency	100 MHz (10 ns)	100 MHz

Table 3: Evaluation of System Performance Metrics after Testing

Flask API latency was not a limiting factor relative to WAN RTT (~tens of ms) and the 200 ms feature cadence; time scales with stored portfolio rows.

In terms of our quantisation error, we had a worst case value higher than our target value. This is mainly due to our revised fixed-point implementation of the matrix multiplier which inherently introduces quantisation error in comparison to our previous float-point implementation. However, we are not concerned by this due to the large number of samples we use, and given that the quantisation error in each sample can be modelled as an independent and identically distributed, we can apply the central limit theorem to our expected error allowing us to represent the error as a zero-mean Gaussian distribution which has a negligible effect on our final output.

Metric	Value
Maximum Multiplication Latency Difference (Software vs Hardware)	68.191x
Average Multiplication Latency Difference (Software vs Hardware)	18.240x
Average Unoptimised Software Latency	8.8573s
Average Optimised Software Latency	30.377ms
Average Hardware Latency	3.4948ms

Table 4: Multiplication Latency Performance Metrics (Software vs Hardware, 5 s.f.)

Hardware Resources (PYNQ-Z1, xc7z020-clg400-1)

Resource	Used	Available	Utilisation
LUT	21,197	53,200	39.84%
FF	21,292	106,400	20.01%
BRAM (36 Kb)	5.5	140	3.9%
DSP48	170	220	77.27%
Total on-chip power			2.135 W
HLS timing (estimated / target)			5.84 ns / 8 ns

Table 5: Table Outlining Hardware Resource Usage

DSP48 utilisation is the highest of any resource at 77.27%. This is expected and intentional: the lower triangle of the 17×17 outer product and all 17 elements of $A^T b$ are fully unrolled into parallel multiply-accumulate units. Using DSPs for every multiplication allow us to have a reduced total cycle count, given all multiplications occur in one cycle. For the lower triangle of $A^T A$, the number of elements required given a dimension size of 17 is 153 (triangular number formula).

6. Evaluation

A linear model trained purely on order book microstructure achieved 58.9% directional accuracy at 250 iterations. That figure settled to around 57.6% at 1417 iterations, suggesting the model’s edge slowly decreases as market microstructure evolves. The paper trading balance grew from \$1000 to \$2126, a 112.6% gain over the test period trading at 20x leverage.

The three-way comparison worked exactly as intended. In the final production runs, the optimised software and hardware weight vectors were consistent to within measurement error, validating the normalisation pipeline. We saw much improved latency when using a Vivado design in comparison to a Vitis HLS design. We also were able to use less resources on the FPGA to build this design. This allowed us to increase our parameters improving our overall linear regression model.

The advantage of hardware acceleration is clear in the accumulation step. The FPGA processes 4096 samples in approximately 3.5ms. The software-optimised NumPy path took 30.4ms for a 4000 sample batch. In practice, the bottleneck is not kernel compute time but the read and write time to DMA.

The overflow and bus-width fixes were both informed by early failures and straightforward once the root cause was understood. The highest-impact extension to this project would be moving the pseudo-inverse computation into the programmable logic via Cholesky or QR decomposition, reducing retrain latency to microseconds. A second PYNQ-Z1 node trading a correlated asset would also demonstrate the multi-node architecture more compellingly.